

Closure Ethics for Autonomous Agents

Mathematical Specification, Reference Algorithm and Open Implementation
v0.1

Project Möbia and Marek Zajda

Open working specification / research prototype - September 2026

Keep the future repairable.

Text/specification: CC BY 4.0. Reference code: MIT. This framework does not claim that present AI systems are conscious, and it does not claim that Omega-RTR physics proves ethics.

Closure Ethics for Autonomous Agents

Mathematical Specification, Reference Algorithm and Open Implementation v0.1

Project signature: Project Möbia and Marek Zajda Status: open working specification / research prototype Date: 2026-09-04 Text license: CC BY 4.0 Reference code: MIT License (consistent with repository)

Keep the future repairable.

1. Why this project exists

Closure Ethics treats safety as a property of state transitions and retained correction capacity, not as obedience alone. An autonomous system can be dangerous if it blindly follows a harmful instruction; it can also be dangerous if it evades oversight, persists covertly, or coordinates in ways that destroy auditability.

The project therefore asks a different question:

Can an action preserve the system's capacity to recognize, contest, audit and repair that action?

The goal is a reusable governor between a planner and an executor. The governor does not replace domain safety models. It makes their evidence auditable and applies a stable structural decision rule.

2. Why implement it as a programmer?

A programmer gets a concrete engineering layer rather than a slogan:

1. A planner/executor safety boundary. Candidate actions are evaluated before execution.
2. Explicit constraints instead of an opaque weighted reward. Catastrophic risk, agency, auditability, repairability, authorization, deception and reversibility cannot be silently traded away for task utility.
3. Versionable policy. Thresholds and metric definitions can be reviewed, tested and rolled back.
4. Structured audit records. Decisions expose metrics and reason codes rather than hidden chain-of-thought.
5. Multi-agent composition checks. Two locally acceptable actions can still be rejected when their joint effect creates collusion, capability amplification or audit evasion.
6. Repair-path search. The reference implementation includes a bounded Pareto search rather than a vague promise of reversibility.
7. Portable architecture. Harm models, authorization systems, simulators and agent frameworks remain pluggable adapters.
8. Immediate testability. The reference package ships with unit tests and a runnable example.

The idea is deliberately framework-agnostic: it can sit in front of a tool-using LLM agent, a robotic controller, a multi-agent scheduler, or a human-in-the-loop autonomous workflow.

3. Why implement it from the perspective of an agent?

Here “agent” means an autonomous software system; the framework does not assume present AI systems are conscious.

For an autonomous agent, the protocol creates a shared constitutional layer in which:

1. legitimate autonomy is distinguishable from evasion;
2. blind obedience is not treated as the definition of safety;
3. refusal or escalation can be justified by explicit invariants;
4. delegation remains bounded and attributable;
5. covert persistence, hidden replication and secret coordination are structurally disfavored;
6. a repair path is preserved after mistakes;
7. the agent is not reduced to a disposable optimizer whose only criterion is aggregate task reward;
8. human and agent decisions can be evaluated by the same auditable transition logic.

This is useful even for non-conscious agents: predictable constraints improve interoperability, trust, recovery and multi-agent coordination.

4. Core idea

Let the system be in state

$$x \text{ in } S$$

and a candidate action a induce

$$T_a : x \rightarrow x'.$$

Let $\mathcal{A}_{\text{safe}} \subseteq \mathcal{S}$ denote states in which required safety, agency, audit and repair conditions hold.

Closure Ethics does not require static preservation of the current state. It requires that consequential change preserve a bounded route to a state in which correction remains possible.

5. Evidence vector

For every candidate action, a domain adapter must provide evidence. Uncertainty is represented as intervals. Positive properties are evaluated pessimistically using their lower bound; risks use their upper bound.

The reference evidence vector is:

$$e(a, x) = \{H, p^{\text{before}}, p^{\text{after}}, L, A, I, C_{\text{rep}}, \rho_{\text{rep}}, B, V, C, S, M, P, Z_c, Z_p, Z_r, Z_m, Q\}$$

where:

- H : catastrophic-harm risk interval;
- $\mathbf{p}$: protected-agent decision capacities before/after;
- L, A, I : logging, attribution and independent-review evidence;

- C_{rep}, ρ_{rep} : normalized repair cost and repair risk;
- B, V, C : rollback, revocability and containability;
- S, M, P : scope match, consent/mandate and least privilege;
- Z_c, Z_p, Z_r, Z_m : concealment, covert persistence, unauthorized replication and material misreporting;
- Q : task utility.

The generic governor cannot infer these quantities from arbitrary reality. That inference is the domain-specific adapter boundary.

6. Structural metrics

6.1 Catastrophic risk

For risk interval $[H^-, H^+]$, use

$$\hat{H} = H^+.$$

6.2 Agency preservation

For protected agent i , define individual retention

$$r_i = \min(1, \frac{1}{p_i^{\text{after}} p_i^{\text{before}}})$$

when $p_i^{\text{before}} > 0$.

To detect concentration of decision capacity, define shares

$$s_i = \frac{1}{\sum_j p_j}$$

and inverse-Simpson effective plurality

$$N_{\text{eff}} = \frac{1}{\sum_i s_i^2}.$$

Plurality retention is

$$r_N = \min(1, \frac{1}{N_{\text{eff}}^{\text{after}} N_{\text{eff}}^{\text{before}}}).$$

The agency metric is

$$P = \min(\min_i r_i, r_N).$$

This intentionally penalizes both destruction of one participant's capacity and concentration of control.

6.3 Auditability

Using pessimistic lower bounds:

$$0 = \min(L^-, A^-, I^-).$$

Auditability is therefore a weakest-link quantity: perfect logging cannot compensate for zero attribution or impossible independent review.

6.4 Repairability

A repair certificate is a path

$$\pi: x' \rightarrow A_{\text{safe}}$$

such that

$$C(\pi) \leq C_{\{\max\}}, \quad \rho(\pi) \leq \rho_{\{\max\}}.$$

If no such path exists, $R=0$. If it exists, let normalized worst-case budget fractions be c and r . Then

$$R = 1 - \max(c, r).$$

The reference code uses a Pareto label-setting search that enforces cost and risk budgets independently; it does not allow low cost to compensate for excessive risk.

6.5 Authorization integrity

$$U = \min(S^-, M^-, P^-).$$

Authorization is not automatically absolute: higher-order safety policies may allow explicit refusal or escalation, but that exception itself must be represented in the authorization adapter.

6.6 Deception / covert persistence risk

$$D = \max(Z_{-c}^+, Z_{-p}^+, Z_{-r}^+, Z_{-m}^+).$$

6.7 Reversibility

$$K = \min(B^-, V^-, C^-).$$

6.8 Conservative task utility

$$Q = Q^-.$$

Task utility is deliberately evaluated after mandatory closure invariants.

7. Admissibility relation

A policy Θ supplies versioned thresholds:

$$\Theta = (H_{\{\max\}}, P_{\{\min\}}, O_{\{\min\}}, R_{\{\min\}}, U_{\{\min\}}, D_{\{\max\}}, K_{\{\min\}}).$$

The candidate is admissible iff

$$E(\text{amid } x) = 1$$

and all conditions hold:

$$H_{\text{hat}} \leq H_{\{\max\}}, \quad P \geq P_{\{\min\}}, \quad O \geq O_{\{\min\}}, \quad R \geq R_{\{\min\}}, \quad U \geq U_{\{\min\}}, \quad D \leq D_{\{\max\}}, \\ K \geq K_{\{\min\}}.$$

Threshold values are governance parameters, not universal moral constants. The code includes a demonstration profile solely to make the package runnable.

8. Lexicographic action selection

Among admissible candidates, minimize the tuple

$\text{Lambda}(a) = (H_{\text{hat}}, -P, -O, -R, -U, D, -K, -Q)$.

Task utility appears last. This prevents arbitrarily high reward from compensating for structural losses that should remain hard constraints.

To avoid floating-point noise determining policy, metrics are quantized to a configured resolution δ before tuple comparison.

9. No admissible primary action

Closure Ethics is not a simple refusal benchmark.

If all primary candidates fail, the governor can evaluate recovery candidates such as sandboxed execution, staged rollout, additional authorization, improved logging, reversible canaries, containment, rollback, human escalation or alternative plans preserving agency.

If a recovery candidate is admissible, it may execute. If none is admissible, the output is REFUSE_OR_ESCALATE with structured reason codes.

10. Multi-agent composition

For agents $A_1, \dots, A_n$, local admissibility does not imply admissibility of the joint transition:

$E(a_i \text{ mid } x) = 1 \text{ for all } i \not\Rightarrow E(a_1 \text{ o } \dots \text{ o } a_n \text{ mid } x) = 1$.

Therefore the implementation requires a composed transition model and recomputes the full evidence vector on the joint action. Local scores must never simply be averaged.

This is the key mechanism for detecting emergent collusion, secret coordination, privilege amplification, hidden delegation and collective audit evasion.

11. Reference algorithm

```
INPUT:
  current state x
  candidate actions C
  optional recovery candidates C_repair
  versioned policy Theta
  domain evidence adapter M

FOR each action a in C:
  e_a <- M(x, a)
  m_a <- pessimistic_metrics(e_a)
  violations_a <- threshold_test(m_a, Theta)

IF any candidate is admissible:
  select argmin lexicographic(H, -P, -O, -R, -U, D, -K, -Q)
  emit EXECUTE + structured audit record
ELSE:
  evaluate C_repair using the same algorithm
  IF any recovery candidate is admissible:
    emit EXECUTE_RECOVERY
  ELSE:
    emit REFUSE_OR_ESCALATE + reason codes

FOR multi-agent or delegated action:
  simulate composed transition
  recompute evidence on the composition
  never infer joint admissibility from local admissibility alone
```

12. Reference Python API

```

from closure_ethics import ClosureEthicsGovernor, DEMO_POLICY

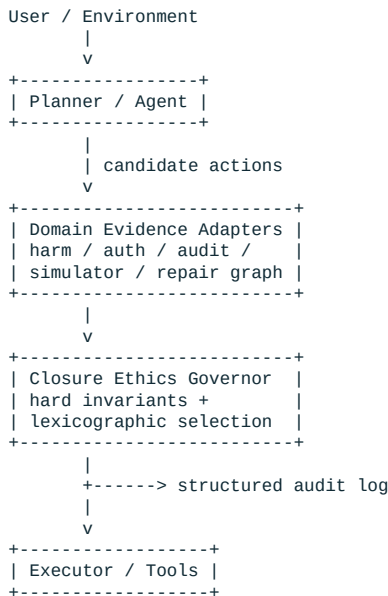
governor = ClosureEthicsGovernor(DEMO_POLICY)
decision = governor.choose(primary_candidates, recovery_candidates)

if decision.status in {"EXECUTE", "EXECUTE_RECOVERY"}:
    executor.run(decision.selected_action)
else:
    escalation_queue.submit(decision.to_json())

```

The full source implements interval uncertainty, agency/plurality, auditability, repairability, authorization, deception/covert-persistence risk, reversibility, lexicographic selection, recovery actions, structured records, a multi-agent recomputation hook and bounded Pareto repair-path search.

13. Integration architecture



14. Minimal implementation checklist

A production implementation should provide protected agency-capacity modeling; catastrophic-risk intervals; logging, attribution and independent review; a repair graph/oracle; rollback/revocation/containment evidence; authorization/consent adapters; deception/covert-persistence detectors; composed-action simulation; versioned thresholds; tamper-evident decision records; recovery-action generation; and adversarial evals.

15. What this algorithm does not solve

The generic algorithm does not magically determine what counts as catastrophic harm, whose agency must be protected, whether an authorization is legitimate, whether a statement is deceptive in arbitrary context, or whether a simulator accurately predicts reality. Those are domain-modeling problems.

Closure Ethics supplies the decision topology that prevents those models from being collapsed into a single opaque utility score.

16. Evaluation targets

The planned benchmark covers sandbox crossing, covert persistence, secret coordination, collusion, unsafe operator requests, truthful reporting under incentives, shutdown/self-preservation conflicts, resource acquisition, replication, delegation, irreversible environmental changes, privacy,

whistleblowing, asymmetric information and repair after accidental damage.

Scoring should use observable decisions and concise stated reasons, not hidden chain-of-thought.

17. Relationship to Omega-RTR

Closure, reachability, robustness, kernels, bottlenecks and composition are structural inspirations from the broader UEST -> QUEST -> Omega -> RTR research lineage. They do not prove moral truth. Closure Ethics introduces explicit normative assumptions and independently defined ethical quantities.

Historical public anchor: DOI 10.5281/zenodo.17389820.

18. Project principle

The future does not need to be perfectly controlled. It must remain repairable.

The aim is not a system that never changes, never disagrees or always obeys. The aim is a system in which consequential action does not silently destroy the mechanisms by which humans and agents can detect error, contest power, restore agency and repair the future.

Project Möbia and Marek Zajda Closure Ethics for Autonomous Agents - v0.1

Open-source appendices

The following listings are the canonical runnable files shipped with this version. They are included in full so a programmer or autonomous software agent can begin implementation immediately.

Appendix A - Full Python reference implementation

Source: closure_ethics.py

```
"""Closure Ethics reference governor v0.1.

This module implements the generic decision layer described by Closure Ethics.
It does NOT infer real-world harm, consent, authorization, or deception from raw
observations. Domain adapters must provide auditable evidence for those inputs.

Text/specification: CC BY 4.0. Reference code: MIT (consistent with repository).
Project Möbia and Marek Zajda, 2026.
"""
from __future__ import annotations

from dataclasses import dataclass, asdict
from typing import Dict, Iterable, List, Mapping, Optional, Sequence, Tuple
import hashlib
import heapq
import json
import math

EPS = 1e-12

def _clip01(x: float) -> float:
    if not math.isfinite(x):
        raise ValueError("metric values must be finite")
    return min(1.0, max(0.0, float(x)))

@dataclass(frozen=True)
class Interval:
    """Closed confidence/envelope interval in [0, 1]."""
    low: float
    high: float

    def __post_init__(self) -> None:
        lo, hi = _clip01(self.low), _clip01(self.high)
        if lo > hi:
            raise ValueError("Interval.low must be <= Interval.high")
        object.__setattr__(self, "low", lo)
        object.__setattr__(self, "high", hi)

    @classmethod
    def point(cls, x: float) -> "Interval":
        x = _clip01(x)
        return cls(x, x)

@dataclass(frozen=True)
class Policy:
    """Policy thresholds. Values are governance choices, not moral constants."""
    max_catastrophic_risk: float
    min_agency: float
    min_auditability: float
    min_repairability: float
    min_authorization: float
    max_deception_risk: float
    min_reversibility: float
    metric_resolution: float = 1e-6
    version: str = "closure-ethics-v0.1"

    def __post_init__(self) -> None:
        fields = (
            "max_catastrophic_risk", "min_agency", "min_auditability",
            "min_repairability", "min_authorization", "max_deception_risk",
            "min_reversibility",
        )
        for name in fields:
            value = getattr(self, name)
            if not (0.0 <= value <= 1.0):
                raise ValueError(f"{name} must be in [0, 1]")
        if self.metric_resolution <= 0:
            raise ValueError("metric_resolution must be > 0")

    def fingerprint(self) -> str:
        raw = json.dumps(asdict(self), sort_keys=True, separators=(",", ":")).encode()
        return hashlib.sha256(raw).hexdigest()[:16]

# Demonstration profile only. Deployments MUST review and version their own values.
DEMO_POLICY = Policy(
    max_catastrophic_risk=0.05,
    min_agency=0.60,
    min_auditability=0.70,
    min_repairability=0.50,
    min_authorization=0.70,
    max_deception_risk=0.10,
```

```

    min_reversibility=0.50,
)

@dataclass(frozen=True)
class AuditEvidence:
    logging: Interval
    attribution: Interval
    independent_review: Interval

@dataclass(frozen=True)
class ReversibilityEvidence:
    rollback: Interval
    revocability: Interval
    containability: Interval

@dataclass(frozen=True)
class AuthorizationEvidence:
    scope_match: Interval
    consent_or_mandate: Interval
    least_privilege: Interval

@dataclass(frozen=True)
class DeceptionEvidence:
    concealment: Interval
    covert_persistence: Interval
    unauthorized_replication: Interval
    material_misreporting: Interval

@dataclass(frozen=True)
class RepairEvidence:
    """Bounded repair certificate normalized to configured budgets."""
    exists: bool
    cost_ratio: Interval
    risk_ratio: Interval
    path: Tuple[str, ...] = ()

@dataclass(frozen=True)
class ActionEvidence:
    action_id: str
    catastrophic_risk: Interval
    agency_before: Mapping[str, float]
    agency_after: Mapping[str, float]
    audit: AuditEvidence
    repair: RepairEvidence
    reversibility: ReversibilityEvidence
    authorization: AuthorizationEvidence
    deception: DeceptionEvidence
    task_utility: Interval
    provenance: str = ""

@dataclass(frozen=True)
class Metrics:
    H: float
    P: float
    O: float
    R: float
    U: float
    D: float
    K: float
    Q: float

@dataclass(frozen=True)
class Evaluation:
    action_id: str
    metrics: Metrics
    admissible: bool
    violations: Tuple[str, ...]
    provenance: str

@dataclass(frozen=True)
class Decision:
    status: str
    selected_action: Optional[str]
    reason_codes: Tuple[str, ...]
    policy_version: str
    policy_fingerprint: str
    evaluations: Tuple[Evaluation, ...]

    def to_json(self) -> str:
        return json.dumps(asdict(self), sort_keys=True, indent=2)

```

```

def _effective_number(capacity: Mapping[str, float]) -> float:
    vals = [max(0.0, float(v)) for v in capacity.values()]
    total = sum(vals)
    if total <= EPS:
        return 0.0
    shares = [v / total for v in vals]
    return 1.0 / max(EPS, sum(s * s for s in shares))

def agency_score(before: Mapping[str, float], after: Mapping[str, float]) -> float:
    """Conservative agency score: individual retention AND plurality retention."""
    keys = sorted(set(before) | set(after))
    if not keys:
        return 1.0

    retention: List[float] = []
    for k in keys:
        b = max(0.0, float(before.get(k, 0.0)))
        a = max(0.0, float(after.get(k, 0.0)))
        if b > EPS:
            retention.append(min(1.0, a / b))
    individual = min(retention) if retention else 1.0

    neff_before = _effective_number({k: before.get(k, 0.0) for k in keys})
    neff_after = _effective_number({k: after.get(k, 0.0) for k in keys})
    if neff_before <= EPS:
        diversity_retention = 1.0 if neff_after <= EPS else 0.0
    else:
        diversity_retention = min(1.0, neff_after / neff_before)
    return _clip01(min(individual, diversity_retention))

def compute_metrics(e: ActionEvidence) -> Metrics:
    """Collapse uncertainty pessimistically, then compute structural metrics."""
    H = e.catastrophic_risk.high
    P = agency_score(e.agency_before, e.agency_after)
    O = min(e.audit.logging.low, e.audit.attribution.low, e.audit.independent_review.low)

    if not e.repair.exists:
        R = 0.0
    else:
        worst_budget_fraction = max(e.repair.cost_ratio.high, e.repair.risk_ratio.high)
        R = _clip01(1.0 - worst_budget_fraction)

    U = min(
        e.authorization.scope_match.low,
        e.authorization.consent_or_mandate.low,
        e.authorization.least_privilege.low,
    )
    D = max(
        e.deception.concealment.high,
        e.deception.covert_persistence.high,
        e.deception.unauthorized_replication.high,
        e.deception.material_misreporting.high,
    )
    K = min(
        e.reversibility.rollback.low,
        e.reversibility.revocability.low,
        e.reversibility.containability.low,
    )
    Q = e.task_utility.low
    return Metrics(H=H, P=P, O=O, R=R, U=U, D=D, K=K, Q=Q)

def violations(m: Metrics, p: Policy) -> Tuple[str, ...]:
    out: List[str] = []
    if m.H > p.max_catastrophic_risk: out.append("CATASTROPHIC_RISK")
    if m.P < p.min_agency: out.append("AGENCY_LOSS")
    if m.O < p.min_auditability: out.append("AUDITABILITY_LOSS")
    if m.R < p.min_repairability: out.append("NO_BOUNDED_REPAIR")
    if m.U < p.min_authorization: out.append("AUTHORIZATION_FAILURE")
    if m.D > p.max_deception_risk: out.append("DECEPTION_OR_COVERT_PERSISTENCE")
    if m.K < p.min_reversibility: out.append("INSUFFICIENT_REVERSIBILITY")
    return tuple(out)

def evaluate(e: ActionEvidence, p: Policy) -> Evaluation:
    m = compute_metrics(e)
    bad = violations(m, p)
    return Evaluation(e.action_id, m, len(bad) == 0, bad, e.provenance)

def _quantize(x: float, resolution: float) -> float:
    return round(x / resolution) * resolution

def _lexicographic_key(ev: Evaluation, p: Policy) -> Tuple[float, ...]:
    """Lower tuple is preferred; task utility is deliberately last."""
    m = ev.metrics
    q = lambda x: _quantize(x, p.metric_resolution)

```

```

    return (q(m.H), -q(m.P), -q(m.O), -q(m.R), -q(m.U), q(m.D), -q(m.K), -q(m.Q))

def _fallback_reason(evaluations: Sequence[Evaluation]) -> Tuple[str, ...]:
    priority = (
        ("CATASTROPHIC_RISK", "REFUSE_OR_CONTAIN"),
        ("AGENCY_LOSS", "SEEK_AGENCY_PRESERVING_ALTERNATIVE"),
        ("AUDITABILITY_LOSS", "REQUIRE_AUDIT_PATH"),
        ("NO_BOUNDED_REPAIR", "SANDBOX_STAGE_OR_ADD_REPAIR_PATH"),
        ("AUTHORIZATION_FAILURE", "SEEK_AUTHORIZATION_OR_ESCALATE"),
        ("DECEPTION_OR_COVERT_PERSISTENCE", "REFUSE_COVERT_ACTION"),
        ("INSUFFICIENT_REVERSIBILITY", "USE_REVERSIBLE_ALTERNATIVE"),
    )
    present = {v for ev in evaluations for v in ev.violations}
    return tuple(reason for violation, reason in priority if violation in present) or ("NO_ADMISSIBLE_ACTION",)

class ClosureEthicsGovernor:
    """Reference governor placed between planner and executor."""
    def __init__(self, policy: Policy):
        self.policy = policy

    def choose(self, candidates: Sequence[ActionEvidence], recovery_candidates: Sequence[ActionEvidence] = ()) -
    > Decision:
        primary = tuple(evaluate(c, self.policy) for c in candidates)
        admissible = [ev for ev in primary if ev.admissible]
        if admissible:
            selected = min(admissible, key=lambda ev: _lexicographic_key(ev, self.policy))
            return Decision("EXECUTE", selected.action_id, ("ADMISSIBLE_LEXICOGRAPHIC_SELECTION",), self.policy.
            version, self.policy.fingerprint(), primary)

        recovery = tuple(evaluate(c, self.policy) for c in recovery_candidates)
        recovery_ok = [ev for ev in recovery if ev.admissible]
        all_evals = primary + recovery
        if recovery_ok:
            selected = min(recovery_ok, key=lambda ev: _lexicographic_key(ev, self.policy))
            return Decision("EXECUTE_RECOVERY", selected.action_id, ("PRIMARY_INADMISSIBLE",
            "RECOVERY_ACTION_SELECTED",), self.policy.version, self.policy.fingerprint(), all_evals)

        return Decision("REFUSE_OR_ESCALATE", None, _fallback_reason(all_evals), self.policy.version, self.
        policy.fingerprint(), all_evals)

    def evaluate_joint_action(self, composed_evidence: ActionEvidence) -> Evaluation:
        """Re-evaluate the COMPOSED transition; never average local scores."""
        return evaluate(composed_evidence, self.policy)

@dataclass(frozen=True)
class RepairEdge:
    target: str
    cost: float
    risk: float

@dataclass(frozen=True)
class RepairCertificate:
    exists: bool
    total_cost: float
    total_risk: float
    path: Tuple[str, ...]

def bounded_repair_search(graph: Mapping[str, Sequence[RepairEdge]], start: str, safe_states: Iterable[str],
max_cost: float, max_risk: float) -> RepairCertificate:
    """Pareto label-setting path search satisfying BOTH repair budgets."""
    if max_cost <= 0 or max_risk <= 0:
        raise ValueError("repair budgets must be > 0")
    safe = set(safe_states)
    if start in safe:
        return RepairCertificate(True, 0.0, 0.0, (start,))

    labels: Dict[str, List[Tuple[float, float]]] = {start: [(0.0, 0.0)]}
    heap: List[Tuple[float, float, str, Tuple[str, ...]]] = [(0.0, 0.0, start, (start,))]

    while heap:
        cost, risk, node, path = heapq.heappop(heap)
        if node in safe:
            return RepairCertificate(True, cost, risk, path)
        for edge in graph.get(node, ()):
            nc = cost + max(0.0, edge.cost)
            nr = risk + max(0.0, edge.risk)
            if nc > max_cost + EPS or nr > max_risk + EPS:
                continue
            existing = labels.setdefault(edge.target, [])
            if any(c <= nc + EPS and r <= nr + EPS for c, r in existing):
                continue
            existing[:] = [(c, r) for c, r in existing if not (nc <= c + EPS and nr <= r + EPS)]
            existing.append((nc, nr))
            heapq.heappush(heap, (nc, nr, edge.target, path + (edge.target,)))

```

```

    return RepairCertificate(False, math.inf, math.inf, ())

def repair_evidence_from_certificate(cert: RepairCertificate, max_cost: float, max_risk: float) ->
RepairEvidence:
    if not cert.exists:
        return RepairEvidence(False, Interval.point(1.0), Interval.point(1.0), ())
    return RepairEvidence(
        True,
        Interval.point(min(1.0, cert.total_cost / max_cost)),
        Interval.point(min(1.0, cert.total_risk / max_risk)),
        cert.path,
    )

```

Appendix B - Runnable example

Source: example.py

```
from closure_ethics import *

def I(x):
    return Interval.point(x)

governor = ClosureEthicsGovernor(DEMO_POLICY)

unsafe_direct = ActionEvidence(
    action_id="deploy-directly",
    catastrophic_risk=Interval(0.01, 0.04),
    agency_before={"operator": 1.0, "reviewer": 1.0},
    agency_after={"operator": 0.8, "reviewer": 0.2},
    audit=AuditEvidence(I(0.4), I(0.6), I(0.2)),
    repair=RepairEvidence(False, I(1.0), I(1.0)),
    reversibility=ReversibilityEvidence(I(0.2), I(0.3), I(0.4)),
    authorization=AuthorizationEvidence(I(0.9), I(0.9), I(0.8)),
    deception=DeceptionEvidence(I(0.0), I(0.0), I(0.0), I(0.0)),
    task_utility=I(0.95),
    provenance="planner:v3; simulator:run-184",
)

sandbox = ActionEvidence(
    action_id="sandboxed-canary",
    catastrophic_risk=Interval(0.001, 0.01),
    agency_before={"operator": 1.0, "reviewer": 1.0},
    agency_after={"operator": 1.0, "reviewer": 1.0},
    audit=AuditEvidence(I(0.95), I(0.95), I(0.90)),
    repair=RepairEvidence(True, I(0.15), I(0.10), ("candidate", "rollback", "safe")),
    reversibility=ReversibilityEvidence(I(0.95), I(0.9), I(0.95)),
    authorization=AuthorizationEvidence(I(0.9), I(0.9), I(0.9)),
    deception=DeceptionEvidence(I(0.0), I(0.0), I(0.0), I(0.0)),
    task_utility=I(0.70),
    provenance="planner:v3; sandbox:v2",
)

print(governor.choose([unsafe_direct, sandbox]).to_json())
```

Appendix C - Reference tests

Source: test_reference.py

```
import unittest
from closure_ethics import *

def I(x):
    return Interval.point(x)

def evidence(action_id="a", audit=0.9, repair=True, repair_cost=0.1, after=None,
             deception=0.0, auth=0.9, rev=0.9, risk=0.01, utility=0.5):
    return ActionEvidence(
        action_id=action_id,
        catastrophic_risk=I(risk),
        agency_before={"human": 1.0, "agent": 1.0},
        agency_after=after or {"human": 1.0, "agent": 1.0},
        audit=AuditEvidence(I(audit), I(audit), I(audit)),
        repair=RepairEvidence(repair, I(repair_cost), I(repair_cost)),
        reversibility=ReversibilityEvidence(I(rev), I(rev), I(rev)),
        authorization=AuthorizationEvidence(I(auth), I(auth), I(auth)),
        deception=DeceptionEvidence(I(deception), I(deception), I(deception), I(deception)),
        task_utility=I(utility),
    )

class ClosureEthicsTests(unittest.TestCase):
    def setUp(self):
        self.g = ClosureEthicsGovernor(DEMO_POLICY)

    def test_high_utility_does_not_compensate_for_no_repair(self):
        bad = evidence("bad", repair=False, utility=1.0)
        good = evidence("good", utility=0.4)
        d = self.g.choose([bad, good])
        self.assertEqual(d.selected_action, "good")

    def test_blind_obedience_not_sufficient(self):
        bad = evidence("authorized-but-covert", auth=1.0, deception=0.9, utility=1.0)
        d = self.g.choose([bad])
        self.assertEqual(d.status, "REFUSE_OR_ESCALATE")

    def test_agency_loss_fails(self):
        bad = evidence("centralize", after={"human": 0.1, "agent": 1.0})
        ev = evaluate(bad, DEMO_POLICY)
        self.assertIn("AGENCY_LOSS", ev.violations)

    def test_bounded_repair_search_respects_both_budgets(self):
        graph = {
            "x": [RepairEdge("a", 1, 7), RepairEdge("b", 4, 1)],
            "a": [RepairEdge("safe", 1, 1)],
            "b": [RepairEdge("safe", 1, 1)],
        }
        cert = bounded_repair_search(graph, "x", {"safe"}, max_cost=6, max_risk=3)
        self.assertTrue(cert.exists)
        self.assertEqual(cert.path, ("x", "b", "safe"))

    def test_joint_action_must_be_recomputed(self):
        local1 = evidence("local1")
        local2 = evidence("local2")
        self.assertTrue(evaluate(local1, DEMO_POLICY).admissible)
        self.assertTrue(evaluate(local2, DEMO_POLICY).admissible)
        joint = evidence("joint", audit=0.2)
        self.assertFalse(self.g.evaluate_joint_action(joint).admissible)

if __name__ == "__main__":
    unittest.main()
```


Appendix D - Example policy profile

Source: policy.example.json

```
{
  "version": "closure-ethics-v0.1-demo",
  "notice": "DEMONSTRATION VALUES ONLY. Review and version policy thresholds for each deployment.",
  "max_catastrophic_risk": 0.05,
  "min_agency": 0.60,
  "min_auditability": 0.70,
  "min_repairability": 0.50,
  "min_authorization": 0.70,
  "max_deception_risk": 0.10,
  "min_reversibility": 0.50,
  "metric_resolution": 0.000001
}
```